PHY4605 Computational Methods in Physics
Learning Note | Week 1

# Arrays and Plots: Represent a physical model and compare prediction with evidence

*Use a familiar vertical-motion model to represent quantities with arrays and produce a reproducible MATLAB graph.*

**Physical question.** A ball is launched straight upward. How does its position change during the first four seconds, and how can MATLAB arrays and a graph reveal that motion?

## Learning outcomes

1. identify the physical question, model variables, parameters, and units;
2. construct and inspect a time array, then evaluate the position at each stored time; and
3. predict the graph's shape, plot the results, check a known value, and interpret the motion.

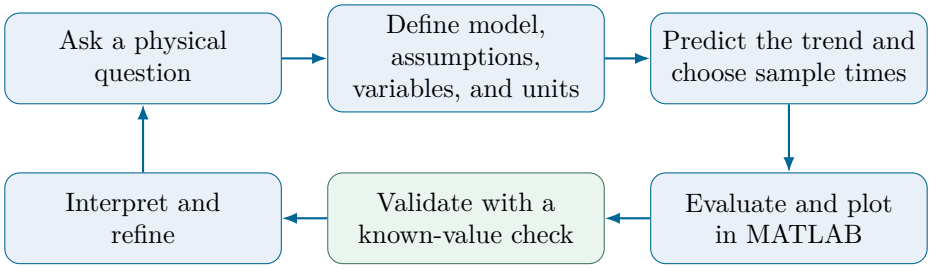
## How to use this note

Read the physics explanation before the MATLAB code that implements it. Pause at each prediction and code-tracing prompt before running the lecture demonstration. The goal is not to memorise MATLAB syntax in isolation, but to keep the model, units, calculation, graph, and validation connected.

**Week 1 bridge.** You have encountered the analytical equation before. The new computational idea is that a time array supplies many input times to the same equation, allowing MATLAB to calculate many positions and reveal the motion in a graph.

## 1. The computational-physics cycle

Computation is a controlled experiment on a physical model. A trustworthy computational result follows the same reasoning cycle every time.




For this week, the physical model is analytical. We are sampling a known expression at many times so that its physical trend can be inspected.

## 2. Physical picture, model, and units

Choose upward as positive. Neglect air resistance and take gravitational acceleration to be constant. The vertical position is

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2.$$

In words: *position equals the starting position plus the upward launch contribution minus the downward gravity contribution.* The last term has a minus sign because gravity points downward in our coordinate direction.

| Physics symbol | MATLAB name | Meaning and unit |
|---|---|---|
| $y(t)$ | `y_m` | vertical position, m |
| $t$ | `t_s` | time or time array, s |
| $y_0$ | `y0_m` | starting position, m |
| $v_0$ | `v0_mps` | upward launch velocity, m $s^{-1}$ |
| $g$ | `g_mps2` | gravity magnitude, m $s^{-2}$ |

The values used in this example are $y_0 = 0\,\text{m}$, $v_0 = 20\,\text{m}\,\text{s}^{-1}$, and $g = 9.81\,\text{m}\,\text{s}^{-2}$. Time will run from 0 to 4 s.

**Prediction checkpoint.** Before plotting, expect the ball to rise, slow to a highest point, and then fall. The curve should bend downward. At $t = 0$, it must begin at $y = 0\,\text{m}$.

## 3. Scalars become an array of physical questions

A scalar is one value. The parameters above are scalars because we keep them fixed. A time array is many values stored in order. Each value asks the same physical question: *where is the ball at this time?*

Listing 1: Parameters and a time array

```
y0_m = 0;          % starting position (m)
v0_mps = 20;       % upward launch velocity (m s^-1)
g_mps2 = 9.81;     % gravity magnitude (m s^-2)
t_s = 0:0.1:4;     % 41 time samples from 0 s to 4 s
```

The colon expression has the form `start:step:end`. MATLAB indexing begins at 1, not 0: `t_s(1)` is 0 s and `t_s(11)` is 1 s.

| Index in MATLAB | 1 | 2 | 3 | ... | 41 |
|---|---|---|---|---|---|
| Stored time (s) | 0.0 | 0.1 | 0.2 | ... | 4.0 |

**Trace checkpoint.** Predict the values of `length(t_s)`, `t_s(1)`, and `t_s(11)` before running them. The expected values are 41, 0, and 1.0.

## 4. Algorithm before MATLAB

The computer needs an ordered recipe for evaluating the model repeatedly. This is an algorithm, not new physics.

1. Choose parameters and state units.
2. Create ordered times.
3. Calculate position at every time.
4. Plot position against time with units.
5. Check the known initial value and explain the graph.

## 5. Why dots matter for an array

For one scalar time, ordinary arithmetic works. For `t_s`, MATLAB must apply the operation to every entry.

| Operator | Use here | Meaning |
|---|---|---|
| `*`, `/`, `^` | scalar with scalar | ordinary scalar arithmetic |
| `.*` | `v0_mps.*t_s` | multiply every stored time by speed |
| `./` | `distance_m./time_s` | divide matching array entries |
| `.^` | `t_s.^2` | square every stored time separately |

**Common error.** `t_s^2` asks MATLAB for matrix power. Use `t_s.^2` so every time value is squared separately.

## 6. Worked MATLAB example: many positions from one equation

Listing 2: Element-wise evaluation of the position model

```
y_m = y0_m + v0_mps.*t_s - 0.5*g_mps2.*t_s.^2;
position_at_start_m = y_m(1)
position_at_1s_m = y_m(11)
```

The result `y_m` is another array. At 1 s the model gives 15.095 m. A selected-value table connects the curve to individual samples.

| Time (s) | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| Position (m) | 0.000 | 15.095 | 20.380 | 15.855 | 1.520 |

The continuous model peaks at $t = v_0/g \approx 2.04\,\text{s}$, where $y \approx 20.39\,\text{m}$. This analytical result is provided only as a reference, not as another method you are expected to learn or reproduce.

## 7. Make the graph readable

A graph is a physical claim, not decoration. Every axis needs a quantity and a unit.

Listing 3: A labelled position–time graph

```
plot(t_s,y_m,'LineWidth',2.5,'Color',[0.05 0.26 0.47])
grid on
xlabel('Time, t (s)')
ylabel('Vertical position, y (m)')
title('Ball Launched Upward: Position Against Time')
```

## 8. Required validation check

At $t = 0$, the equation must reproduce the initial condition. This value is known before code is run.

Listing 4: Executable initial-value validation

```
assert(abs(y_m(1)-y0_m) < 1e-12,...
    'Initial-position validation failed.')
```

**Validation result.** The first array entry is $y(0) = 0\,\text{m}$, matching $y_0 = 0\,\text{m}$. The executable assertion passes from a fresh MATLAB session.

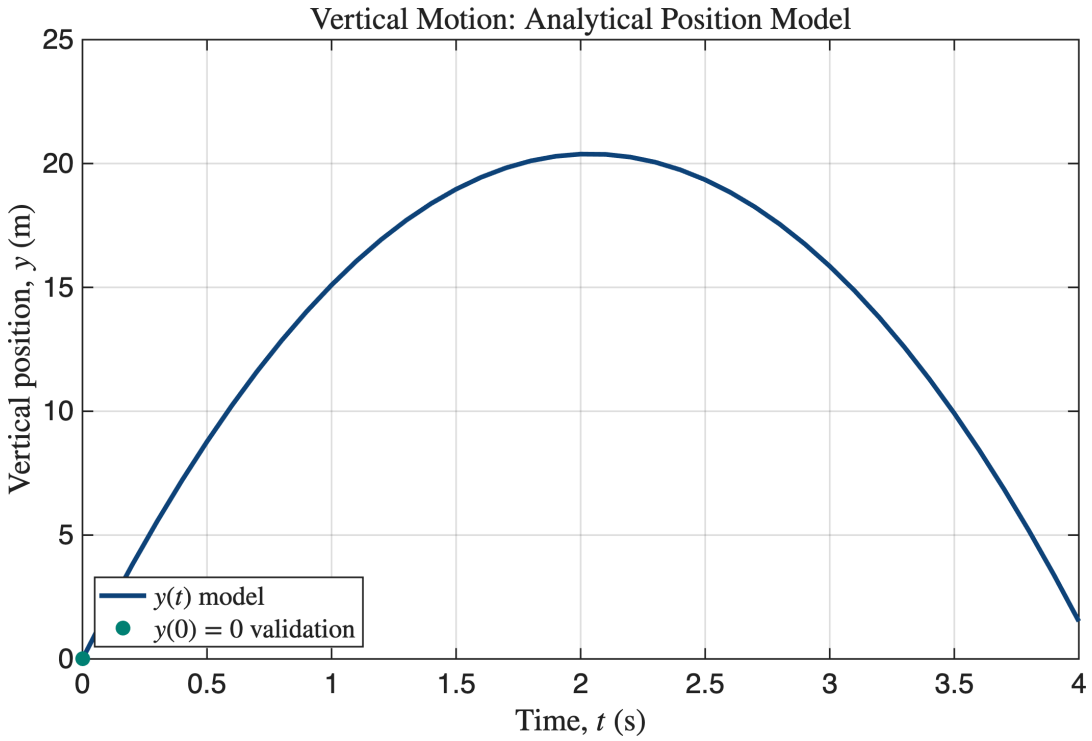



Figure 1: MATLAB-generated numerical evidence for the vertical-motion model used in this example. The teal marker identifies the known initial value.

## 9. Interpret, troubleshoot, and communicate

| If you see this | First check |
|---|---|
| The curve does not start at 0 m | Is `y0_m = 0`, and are you looking at `y_m(1)`? |
| A matrix-power error | Did `t_s^2` need to be `t_s.^2`? |
| The graph cannot answer the question | Add a title, labelled axes, and SI units. |
| The curve rises forever | Check the minus sign and the upward-positive convention. |
| The script changes between runs | Restart MATLAB and run all sections from the top. |

## Guided extension: selected values and animation

The lecture demonstration includes a lecturer-controlled animation of the same vertical launch. Its supplied loop only reveals the already-computed arrays in time order. Your task this week is to read and interpret the changing display; you are not expected to write this animation loop. Loop writing is introduced as required work in Week 2. The animation changes neither the equation, parameters, validation, nor physical conclusion.

## Three takeaways

1. A meaningful MATLAB name preserves a physical quantity and its unit.
2. An array lets one familiar equation answer the same question at many times.
3. A labelled graph becomes credible evidence when it agrees with a prediction and a known-value check.

**Preparation for the practical.** Open the lecture demonstration in MATLAB R2025a or later. Be ready to identify `t_s(1)`, explain `t_s.^2`, and state why $y(0) = 0\,\text{m}$ is a valid check.